Αρχιτεκτονική Προηγμένων Υπολογιστών και Επεξεργαστών

Flynn's Taxonomy (ταξινόμηση)

- **SISD** - Single Instruction Single Data Stream — Uniprocessor
- **SIMD** - Single Instruction Multiple Datastreams — Vector architectures
- **MISD** - Multiple Instructions Single Datastream — No commercial multiprocessor of this type built
- **MIMD** - Multiple Instructions Multiple Data Streams — Multiple vector instructions executed

Dependability

- **MTTF** - Mean time to failure
- **MTTR** - Mean time to repair
- **MTBF** - Mean time between failures — $MTBF = MTTF + MTTR$

$$Availability = \frac{MTTF}{MTBF}$$

Example

10 disks : 1 000 000 h MTTF
1 PSU : 200 000 h MTTF

$$Failure\ rate_{system} = 10 \cdot \frac{1}{1000000} + \frac{1}{200000} = \frac{15}{1000000}\ [1/h]$$

$$MTTF_{system} = \frac{1}{Failure\ rate_{system}} = 66666,\bar{6}\ [h]$$

Amdahl's Law (page 49)

The performance gain that can be obtained by improving some portion of a computer.

$$Execution\_time_{NEW} = Execution\_time_{OLD} \times \left((1 - fraction_{enhanced}) + \frac{fraction_{enhanced}}{speedup_{enhanced}} \right) \Rightarrow$$

$$\Rightarrow Speedup_{overall} = \frac{Execution\_time_{OLD}}{Execution\_time_{NEW}} = \frac{1}{(1 - fraction_{enhanced}) + \frac{fraction_{enhanced}}{speedup_{enhanced}}}$$

Example

CPU_{NEW} is 10 times faster on computation

CPU_{OLD} was busy with computation 40% of the time

What's the speedup?

$fraction_{enhanced} = 0,4$

$$speedup_{overall} = \frac{1}{\underbrace{(1-0,4)}_{\text{unaffected percentage}} + \underbrace{\frac{0,4}{10}}_{\text{affected percentage after speedup}}} = \frac{1}{0,64} \approx 1,56$$

The Processor Performance Equation (page 52)

$$\text{CPU time} = \frac{\text{CPU clock cycles}}{\text{clock rate}} = \text{CPU clock cycles} \times \text{Clock cycle time}$$

clock rate: Hz

$$\text{CPI} = \frac{\text{CPU clock cycles}}{\text{Instruction count}}$$

$$\text{CPU time} = \text{CPI} \times \text{Instruction count} \times \text{Clock cycle time}$$

$$\text{CPU clock cycles} = \sum_{i=1}^{n} IC_i \times CPI_i$$

sum:
- $\text{Instruction Count}_1 \times \text{CPI}_1$
- $\text{Instruction Count}_2 \times \text{CPI}_2$
- ⋮

different types of instructions

Bandwidth vs Latency

(data throughput) (data response time)

1 job done in 10 seconds
- latency → 10 [seconds]
- bandwidth → $\frac{1}{10}$ [jobs/seconds]

This is true only when there is no parallel computation:

$$\text{latency} = \frac{1}{\text{bandwidth}}$$

Cache Memory

Index + Offset + Tag

Offset: Determines the specific byte within a cache block.

Index: Select which Direct Mapped Cache line / Set-Associative Cache set we are examining

Tag: Validate if the data stored in the cache line correspond to the main memory address we are trying to access

V	D	Tag	Data

Cache line / Cache Block

V: Valid

Indicates whether a cache line is free "storing garbage" or storing a valid block of memory.

D: Dirty

Indicates whether the data stored in a cache line has been written.

Used by write-back caches to signify that a cache block differs from its main memory counterpart (out of sync)

Block replacement
- Random
- LRU (Least Recently Used)
- FIFO (First in first out)

Cache Designs

Direct-Mapped

Every block of main memory maps to exactly one cache line.

Fully Associative

Any block of main memory can map to any cache line

Set-Associative

Every block of main memory maps to one cache set.

Write-through vs Write-back cache (write mechanisms)

always syncs with main memory	allows to safely go out of sync
simplicity	performance

Write-allocate: If data not in cache bring it from storage. ✓ utilizes space/time locality
No-write-allocate: If data not in cache write directly to storage skipping cache ✗

Cache Misses

Compulsory - Cold start: the need to load the blocks from the main memory in the start

Capacity - The cache is unable to contain all blocks accessed by the program

Conflict - Multiple memory locations mapped to the same cache location (Set-Associative and Direct-Mapped)

Coherence - An other process updates memory

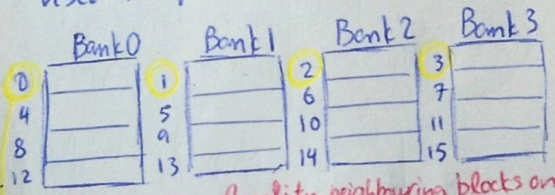
Fully Associative can't exhibit conflict misses.

Average Memory Access Time (AMAT) = Hit time + (Miss Rate × Miss Penalty)

= (Hit Rate × Hit Time) + (Miss Rate × Miss Time)

Multi-banked Caches

Used for parallel access



utilize space locality, neighbouring blocks on different banks (parallel read)

Non-blocking Caches

Allow hits before previous misses complete.

- hit under miss
- hit under multiple misses

Non-blocking caches effectively reduce the miss penalty by overlapping execution with memory access.

Hit time, Miss rates etc Tables (pages 117, B-40)

Branch Prediction

- 2-bit predictor
- Correlating predictor
- Tournament predictor

uses the behaviour of the last m branches

(m,n) branch predictor

2^m n-bit branch predictor to choose from 2^m n-bit predictors

① 2-bit predictor

0	0
0	1
1	0
1	1

Only changes its prediction when it is proven wrong 2 times in a row

If the prediction is wrong two consecutive times, change prediction

② Correlating predictor (two level predictor)

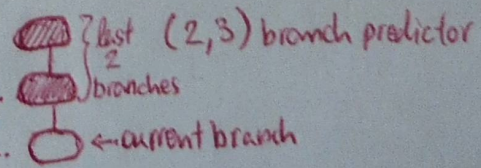
- Uses additional branch predictors from nearby branch instructions to raise accuracy.

n = number of missed predictions to change prediction

③ Tournament predictor

- Global branch predictor table
- Local branch predictor table

Another branch predictor chooses between local-global.



1 2-bit branch predictor for each branch instruction in our program

Register Renaming

Part of dynamic scheduling: rearrange order of instructions to reduce stalls while maintaining data flo

```
fdiv.d  f0, f2, f4
fmul.d  f6, f0, f8
fadd.d  f0, f10, f14
```

↳ Rename to e.g. f1 so that the program doesn't stall (waiting for the multiplication to finish)

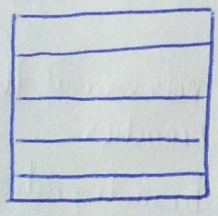
Anti-dependance

f0 is an antidependance, a dependance that is not real

Speculative Execution

Execute instructions along predicted execution paths but only commit the results if prediction was correct.

Reorder Buffer



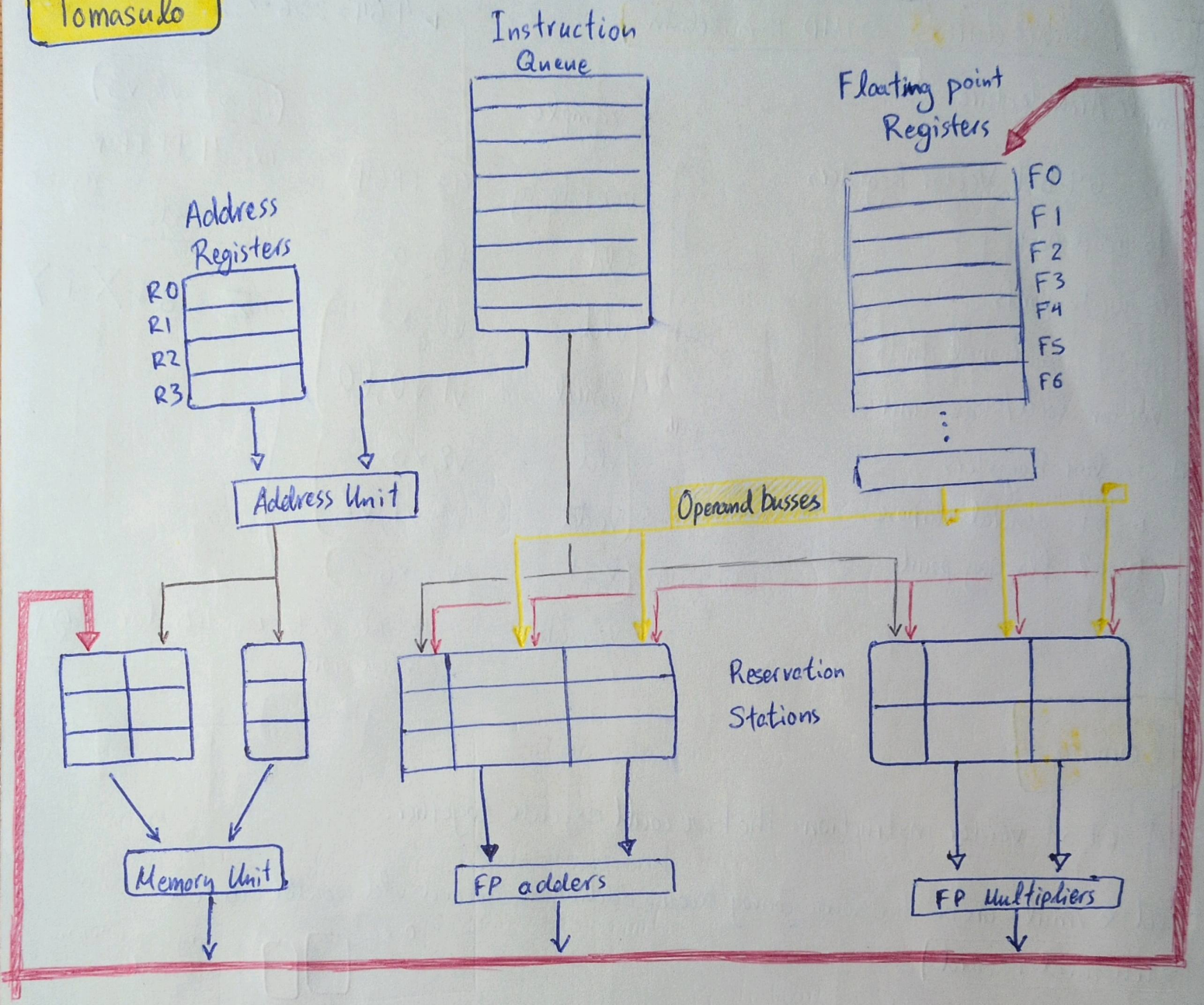
} holds the result of instruction between completion and commit.

Instruction commit: allows instructions to update the register file when the instruction is proven non-speculative.

Data Hazards

- Read after Write (RAW) - (trying to read a register before an earlier instruction writes to it)
- Write after Write (WAW) - (writing to a register after a later write instruction)
- Write after Read (WAR) - (writing before an earlier read instruction has finished)

Tomasulo



Instruction status

Iteration	Instruction	j	k
1	LD F0	0	R1
1	MULTD F4	F0	F2
1	SD F4	0	R1
1	LD F0	0	R1
2	MULTD F4	F0	F2
2	SD F4	0	R1

	Issued	Completed	Result
Load1			
Load2			
Load3			
Store1			
Store2			
Store3			

	Busy	Address
Load1		
Load2		
Load3		
Store1		
Store2		
Store3		

Reservation Stations

Time	Name	Busy	Op	Vj	Vk	Qj	Qk
	Add1						
	Add2						
	Add3						
	Mult1						
	Mult2						

Code

```
LD    F0  0  R1
MULT  F4  F0 F2
SD    F4  0  R1
SUBI  R1  R1 #8
BNEZ  R1  Loop
```

Register result status

Clock

	F0	F2	F4	F6	F8	F10	F12	...	F30
FU									

Function units

Vector Architectures - SIMD Parallelism

Example Architecture

- 32 64-bit Vector Registers
- 16 read ports
- 8 write ports
- Vector functional units
- Vector load/store unit
- Scalar Registers
  - 31 general purpose
  - 32 floating point

Example

4 * 64 = 256 ⇒ 256/8 = 32 words

v0, v1, v2, v3

vsetdcfg 4*FP64 } enable 4 DP FP vector registers

load a { fld f0, a

load vector X { vld v0, x5

multiply vector X with scalar a { vmul v1, v0, f0

load vector Y { vld v2, x6

Z = aX + Y { vadd v3, v1, v2

store vector Z { vst v3, x6

vdisable } disable vector registers for energy consecution

C code

$Z = a \times X + Y$

Convoy

A set of vector instructions that we could execute together.

vld & vmul are on the same convoy because adding the second vld would create a **structural hazard**.

e.x. 2 large operations parallelized

explained

vld
vmul

and

vld
vadd

→ Can exist in the same convoy because of "chaining"

vld v0, x5
vmul v1, v0, f0

once the first element of v0 has been loaded it immidiately used by vmul to calculate the first element of v1. This is called

Chime: unit of time to execute one convoy

1st: vld, vmul
2nd: vld, vadd
3rd: vst

} 3 chimes

cycles per FLOP (floating point operation) = 1,5

2 floating point operations per result

- 1 multiply
- 1 add

32 κύκλοι για κάθε πράξη × 3 chimes = 96 κύκλοι

Scatter-Gather : Allows programs with sparse matrices to execute in vector mode.

```
for (i=0;i<n;i++)
    A[K[i]] = A[K[i]] + C[M[i]]
```

```
vsetdcfg   4*FP64   } enable 4 64-bits floating point registers
vld        v0,x7    } load K                    → A[]
vldx       v1,x5,v0 } load A[K[]]               → offset
vld        v2,x28   } load M
vldi       v3,x6,v2 } load C[M[]]
vadd       v1,v1,v3 } A[K[]] + C[M[]] = A[K[]]
vstx       v1,x5,v0 } store A[K[]]
vdisable   } disable vector registers
```

load vector indexed ↓ "gather"

Opposite is vsti: store vector indexed "scatter"

vldi / vldx } same thing
vsti / vstx }

"Nor Vector length is larger or equal to n. Otherwise we would have to use vector length register (setvl)." !

vldi vsti encodes format:

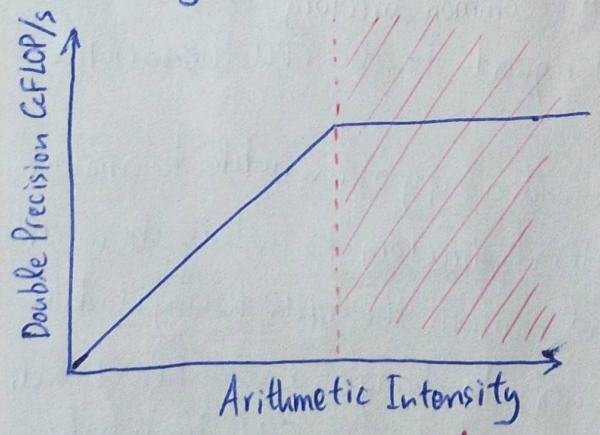
vldi v1, x5 (v0)

- v1 → register holding the loaded value
- x5 → memory location
- (v0) → offset

Roofline Performance Model — An upper bound on performance of a kernel.

Floating point throughput as a function of arithmetic intensity.

Arithmetic intensity: FLOP / byte read (floating point operations)



: Code uses
- the full main memory bandwidth
- the full computational performance of the chip

↓ memory bottleneck
e.g. Badly optimized code, Bad architecture

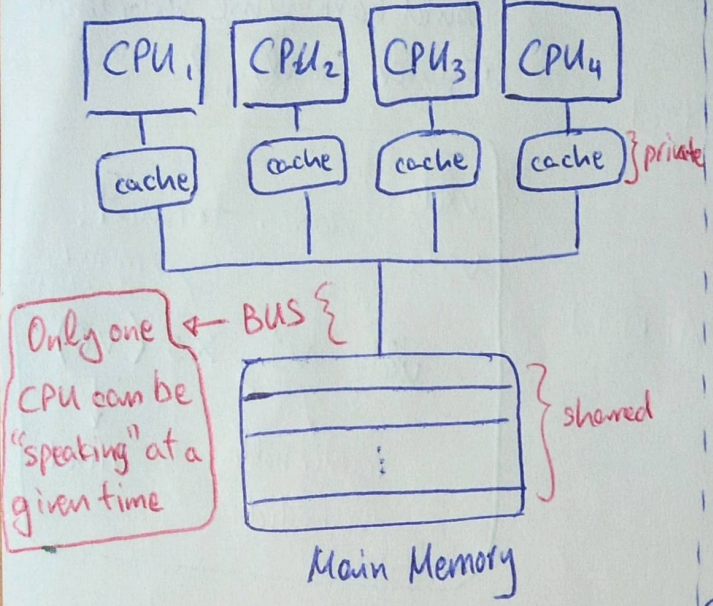
↓ computational bottleneck

protocols for cache-coherence

Snooping-based and Directory-based Coherency Protocol

distributed

Snooping Each core tracks sharing status of each block



- When a CPU obtains a variable it becomes SHARED.
- If another CPU obtains the same variable it is fetched from the main memory and set to SHARED.
- If one of the CPUs change the variable it sets the variable status to MODIFIED and sends an INVALIDATE SIGNAL to all other caches to invalidate their local copy of the variable Status: SHARED → INVALID
- Once all instances of the variable get updated their status changes to SHARED.

Con

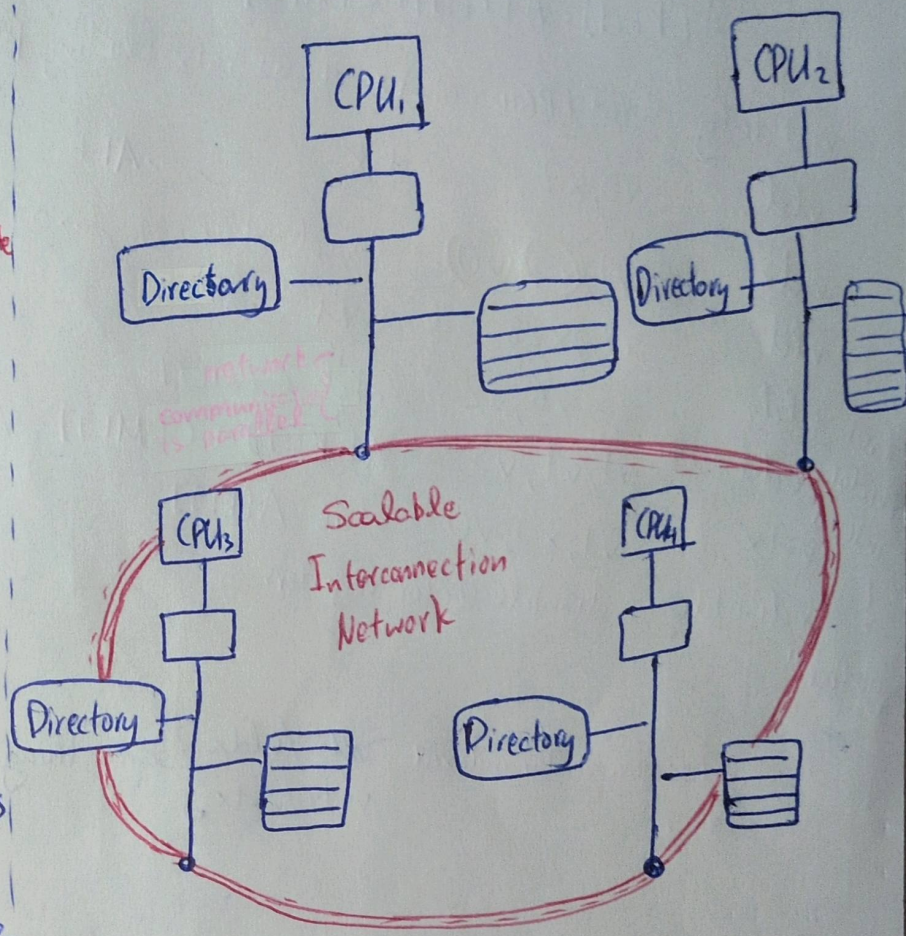
- If there are many CPUs then the many CPU broadcasts lead to common shared bus contention and therefore a bottleneck.

write invalidate: The other CPUs have to fetch the invalid variable from the main memory.

write update: The local copies of the other CPUs are updated instead of invalidete and don't have to search the main memory.

centralized

Directory Sharing status of each block kept in one location



- Mainly meant for distributed systems
- If a CPU wants a variable, it is informed of its location from the common directory. Then it sends a point to point request to the CPU the variable is in.
- When a CPU send a copy of a variable to another CPU, it updates its directory to feature the variable as SHARED and mentions the CPUs having its data.
- When a CPU wants to change a variable it sends an INVALIDATE signal to all CPUs that hold the same data. It knows using the directory.
- When the other CPU INVALIDates its copy, it send an ACKnowledgement signal.
- Then the CPU that changed the variable remaves the other CPU from its directory since it no longer holds valid data.

Tomasulo Examples

Tomasulo Loop Example

```
Loop: ld    F0   0   R1
      multd F4   F0  F2
      sd    F4   0   R1
      subi  R1   R1  #8
      bnez  R1   Loop
```

- loads take 8 clocks (on cache miss)
- multiply takes 4 clocks
- loads take 4 clocks (on cache hit)
- stores take 4 clocks (write through)

Given loop unrolling by a factor of 2:

Issue	Execution completed	Write result	Instruction	
1	9	10	ld F0, 0, R1	iteration 1
2	14	15	mult F4, F0, F2	
3	19	20	sd F4, 0, R1	
6	10	11	ld F0, 0, R1	iteration 2
7	15	16	mult F4, F0, F2	
8	20	21	sd F4, 0, R1	

Θέλει 4 clocks πρέπει να γίνουν subi και bnez

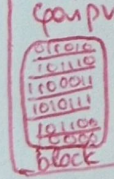
Για να γίνουν το issuing

1
2
3
4
...

Θα έπρεπε ο compiler να βάλει manually ld F0, -8, R1

Εδώ αποφεύγω το subi R1, R1, #

Φαίρνοντας την διεύθυνση μνήμης R1 φαίρνει και όλο το block της



Εδώ στο επόμενο iteration δεν θα έχαμε hit miss. (R1-8 βρίσκεται εντός αυτού)

Tomasulo simple example

	issue	complete	write
ld F6, 34+, R2	1	3	4
ld F2, 45+, R3	2	4	5
mult F0, F2, F4	3	15	16
subd F8, F6, F2	4	7	8
divd F10, F0, F6	5	56	57
addd F6, F8, F2	6	10	11

5+10 cycles for mult

5+2 cycles for addition/subtaction

8+2 cycles for addition

- division takes 40 cycles
- loads take 2 cycles
- addition takes 2cycles
- multiplication takes 10 cycles
- 3 load reservation stations
- 3 add reservation stations
- 2 mult reservation stations

Finally the division completes at cycle 56 = 16 + 40 cycles for completion

cycle in which F0 was given

Scoreboard

outdated not on exams

(same with Tomasulo simple example notespage 10)

Example

	issue	read operands	execution complete	write result
ld F6, 34+, R2	1	2	3	4
ld F2, 45+, R3	5	6	7	8
multd F0, F2, F4	6	9	19	20
subd F8, F6, F2	7	9	11	12
divd F10, F0, F6	8	21	61	62
addd F6, F8, F2	13	14	16	22

Op	takes	cycles
divd	→	40
multd	→	10
addd	→	2
subd	→	2

The last addd starts on cycle 13 because it waits for the subd to finish, freeing the adder unit.

F2 from the second load is needed to read the multd and subd operation

Functional units:

- 2 multipliers
- 1 adder
- 1 divider
- 1 integer (for loads)

needs to wait for the first load since I have only one integer unit

In Tomasulo the read operands stage doesn't exist. This is because the values are either loaded during the issue phase (1) or via the reservation stations (2) when the value gets broadcasted through the Common data bus (CBD).

Tomasulo (is a smart HARDWARE solution which:)

- Does register renaming
- Doesn't have to wait for values to get written and then read from the register file
- Utilizes the common data bus (CDB) to broadcast results to instructions waiting for them

Disadvantage

- CBD bottleneck → Solution: Register Renaming via virtual registers

Multiple Issue

The Ability to start multiple instructions on the same cycle.

		Hazard detection	
1	Superscalar (static)	HW	Static scheduling, in order execution
2	Superscalar (dynamic)	HW	Dynamic Scheduling, some out of order execution
3	Superscalar (superlative)	HW	Dynamic Scheduling w/ speculation, out of order execution
4	VLIW/LIW	SW	Very long Instruction Word (static scheduling, all hazards determined by compiler) The CPU executes one instruction which consists of multiple instructions
5	EPIC	SW	

VLIW

statically finding parallelism

The compiler packs instructions in groups of 5. These instructions can be executed together. → or whatever the computer architect decided when building the processor

example:	memory reference 1	memory reference 2	FP operation 1	FP operation 2	Intiger operation
group 1	fld f0, 0(x1)	fld f6, -8(x1)			
group 2	fld f10, -16(x1)	fld f14, -24(x1)			
group 3	fld f18, -32(x1)	fld f22, -40(x1)	fadd.d f4, f0, f2	fadd.d f8, f6, f2	
group 4	fld f26, -48(x1)		fadd.d f12, f0, f2	fadd.d f16, f14, f2	
group 5			fadd.d f20, f18, f2	fadd.d f24, f22, f2	
group 6	fsd f4, 0(x2)	fsd f8, -8(x1)	fadd.d f28, f26, f24		
group 7	fsd f12, -16(x1)	fsd f16, -24(x1)			addi x1, x1,
group 8	fsd f20, 24(x1)	fsd f24, 16(x1)			
group 9	fsd f28, 8(x1)				bne x1, x2, lop

Two-Issue Processor

Code:

```
Loop: ld   x2, 0(x1)
      addi x2, x2, 1
      sd   x2, 0(x1)
      addi x1, x2, 8
      bne  x2, x2, Loop
```

Without Speculation

iter	Instruction	issue	execute	memory access	write (CDB)
1	ld x2, 0(x1)	1	2	3	4
1	addi x2, x2, 1	1	5		6
1	sd x2, 0(x1)	2	3	7	
1	addi x1, x1, 8	2	3		4
1	bne x2, x3, Loop	3	7		
2	ld x2, 0(x1)	4	8	9	10
2	addi x2, x2, 1	4	11		12
2	sd x2, 0(x1)	5	9	13	
2	addi x1, x2, 8	5	8		9
2	bne x2, x3, Loop	6	13		
3	ld x2, 0(x1)	7	14	15	16
3	addi x2, x2, 1	7	17		18
3	sd x2, 0(x1)	8	15	19	
3	addi x1, x1, 8	8	14		15
3	bne x2, x3, Loop	9	19		

load executes at cycle 8 since I need to wait for the branch's result

With Speculation

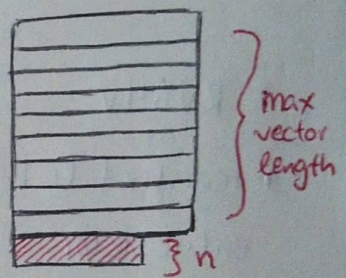
iter	Instruction	issue	execute	read access	write (CDB)	commit
1	ld x_2, 0(x_1)	1	2	3	4	5
1	addi x_2, x_2, 1	1	5		6	7
1	sd x_2, 0(x_1)	2	3			7
1	addi x_1, x_1, 8	2	3		4	8
1	bne x_2, x_3, Loop	3	7			8
2	ld x_2, 0(x_1)	4	5	6	7	9
2	addi x_2, x_2, 1	4	8		9	10
2	sd x_2, 0(x_1)	5	6			10
2	addi x_1, x_1, 8	5	6		7	11
2	bne x_2, x_3, Loop	6	10			11
3	ld x_2, 0(x_2)	7	8	9	10	12
3	addi x_2, x_2, 1	7	11		12	13
3	sd x_2, 0(x_1)	8	9			13
3	addi x_1, x_1, 8	8	9		10	14
3	bne x_2, x_3, Loop	9	13			14

Vector Length Register

Code in C:

```
for (i=0, i<n, i=i+1){
    Y[i] = a * X[i] + Y[i]
}
```

```
vsetdcfg  2DPFP   #enable 2 64bit floating point registers
fld       f0, a   # load a (scalar)
loop: setvl  t0, a0   # set the vector length = min(mvl, n)   { mvl: max vector length
                                                               { n: last elements in final iteration
      vld    v0, xs   # load X[] (vector)
      slli   t1, t0, 3   # vector length *8
      add    xs, xs, t1  # increment pointer to X[] (by 8 bytes = size of DP FP register)
      vmul   v0, v0, f0  # a*X[]
      vld    v1, x6     # load Y[] (vector)
      vadd   v1, v0, v1  # Y[] = a*X[] + Y[]
      sub    a0, a0, t0  # n=n-vl
      vst    v1, x6     # store Y[]
      add    x6, x6, t1  # increment pointer to Y[] (by 8 once again)
      bnez   a0, loop   # if n=0 end loop
      vdisable  # disable vector registers
```



Vector Mask Register

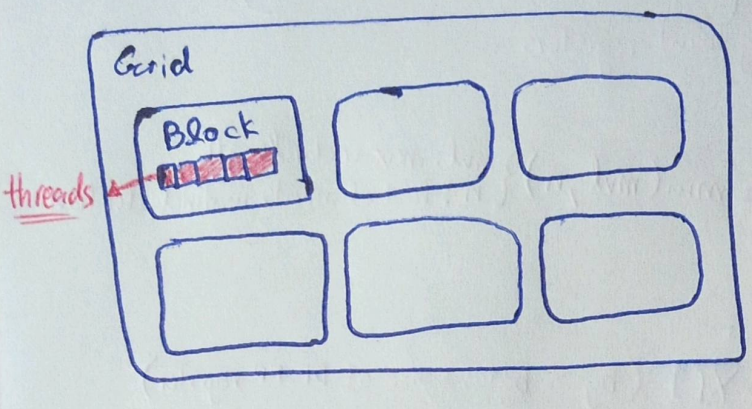
Code in C:

```
for(i=0, i<64, i=i+1){
    if(X[i] != 0){
        X[i] = X[i] - Y[i]
    }
}
```

```
vsetdcfg   2*FP64  #enable 2 64bit floating point registers
vsetpcfgi  1       #enable 1 predicate register
vld        v0, x5  # load X[i]
vld        v1, x6  # load Y[i]
fmv.d.x    f0, x0  # f0 = 0
vpne       p0, v0, f0  # P0 = 1 if X[] != 0
vsub       v0, v0, v1  # X[] = X[] - Y[]
vst        v0, x5  # store X[]
vdisable   # disable vector registers
vpdisable  # disable predicate registers
```

"Executes only for the elements where X[] != 0 (or, we could say, where P0=1)"

GPUs (most likely not on the exam)



A block is assigned to a multithreaded SIMD processor by the thread block scheduler.

Warp: A set of parallel CUDA Threads that execute the same instruction together in a multithreaded SIMT/SIMD processor

Single Instruction Multiple Thread

Single Instruction Multiple Datastreams

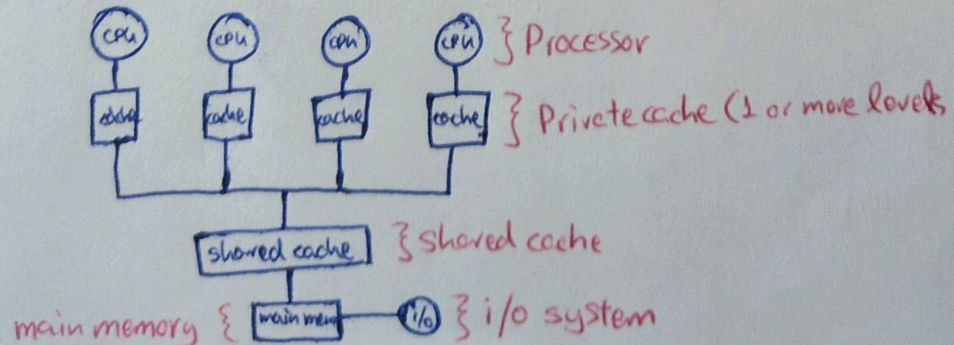
Vector Architecture vs GPUs

RV64V	GPU
register file holds entire vectors	distributes vectors across the registers of SIMD lanes
32 vector registers of 32 elements (1024)	256 registers with 32 elements each (8192)
2-8 lanes with vector length of 32; chime is 4-16 cycles	SIMD processor chime is 2-4 cycles

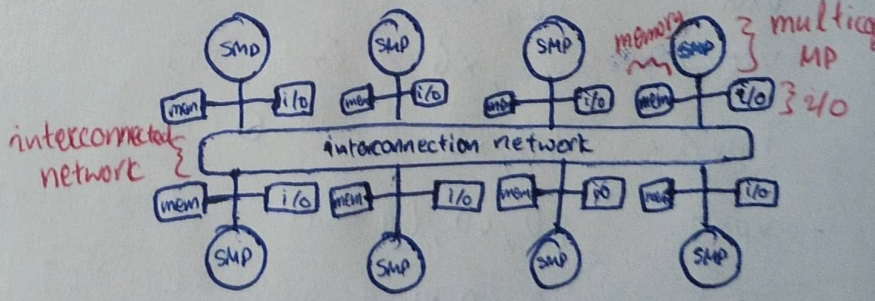
Thread-Level Parallelism

Types

Symmetic multiprocessors (SMP)



Distributed shared memory (DSM)

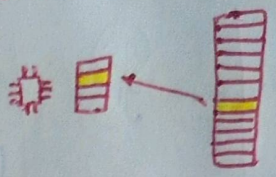


DSMs are many SMPs connected through a fast interconnection network.

Aka NUMA (non uniform memory access)

Cache coherence protocols recap

Data characterization (MSI) or (MESI) Local cache block which corresponds with the block I want to find in memory.



Invalid: The value of the local block is invalid.

Shared: I share this memory block with the other CPUs.

Modified: I modified the memory block.

Exclusive: I am the only holder of this memory block.

Snooping

Request	Source	State of cache block	Action	Explanation
Read hit	Processor	Shared or Modified	Normal hit	Read data in local cache
Read miss	Processor	Invalid	Normal miss	Place read miss on bus
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus
Read miss	Processor	Modified	Replacement	Address conflict miss: write back block then place read miss on bus
Write hit	Processor	Modified	Normal hit	Write data in local cache
Write hit	Processor	Shared	Coherence	Place invalidate on bus
Write miss	Processor	Invalid	Normal miss	Place write miss on bus
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus
Write miss	Processor	Modified	Replacement	Address conflict miss: write back block then place write miss on bus
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss
Read miss	Bus	Modified	Coherence	Attempt to read shared data: place cache block on bus, write back block and change state to shared
Invalidate	Bus	Shared	Coherence	Attempt to write shared block: invalidate the block
Write miss	Bus	Shared	Coherence	Attempt to write shared block: invalidate the cache block
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere: write back the cache block and make its state invalid in the local cache

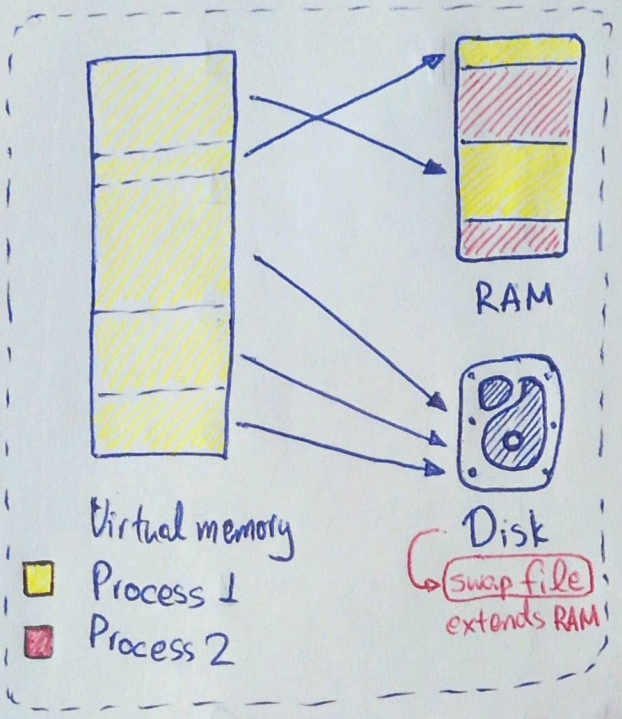
Directory

Message type	Source	Destination	Message contents	Explanation
Read miss	Local cache	Home directory	P, A	Node P has read miss at address A; request data and make P a read sharer
Write miss	Local cache	Home directory	P, A	Node P has write miss at address A; request data and make P the exclusive owner
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A
Invalidate	Home directory	Remote cache	A	Invalide a shared copy of data at addess A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared
Fetch/Invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache
Data value reply	Home directory	Local cache	D	Return a data value from the home memory
Data write back	Remote cache	Home directory	A, D	Write back a data value for address A.

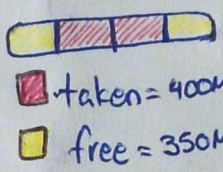
Virtual Memory

"A virtual address space mapped to physical memory."

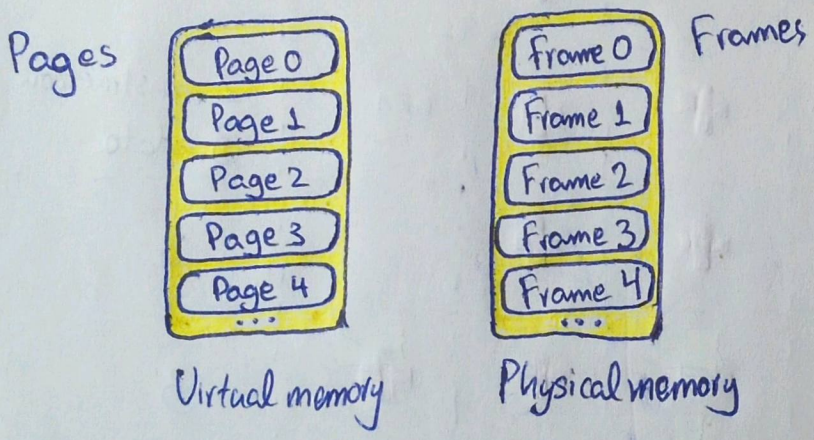
- Helps run larger programs than there is memory available.



- Without it processes could access the memory of other processes. This is a big security issue.
- Solves fragmentation utilizing the full memory.



Even though this program could fit since 350>320 when using direct memory access no individual fragment can fit the whole program so it is unable to load.



The pages are loaded to frames regardless if they are continuous or not.

The TLB translate the virtual page into a physical frame. It works like a cache retaining some recent mappings of pages to frames.

Page Number → which page

Page Offset → which byte

RAM types

- SRAM → cache
- DRAM → RAM
- SDRAM → RAM with its own small cache

6 ways to make cache better

Cache size stays the same so larger block size means less blocks

1. Larger block size
   - ↓↓ compulsory misses
   - ↑↑ capacity, conflict misses
   - ↑↑ miss penalty (more data to carry)
2. Larger total cache capacity
   - ↑↑ hit time, power consumption
   - ↓↓ miss rate
3. Higher Associativity
   - ↑↑ hit time, power consumption
   - ↓↓ conflict misses
4. Higher number of cache levels
   - ↓↓ overall memory access time
5. Prioritize read misses over writes
   - ↓↓ miss penalty
6. Avoid address translation in cache indexing
   - ↓↓ hit time

Technique	Hit time	Bandwidth	Miss penalty	Miss rate	Power consumption	Hardware cost / Complexity
Small & simple caches	+			−	+	0
Way-predicting caches	+				+	1
Pipelined-banked caches	−	+				1
Non-blocking caches		+	+			3
Critical word first and early restart			+			2
Merging write buffer			+			1
Compiler techniques				+		0
Hardware prefetching of instructions & data			+	+	−	2 instruction 3 data
Compiler controlled prefetching			+	+		3
HBM as additional level of cache		+ / −	−	+	+	3

+ improves the factor
− hurts the factor

easiest 0 ... 3 challenge

the software variant does the same thing by writing new loads

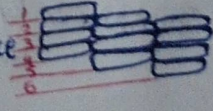
Hardware prefetching: fetch multiple blocks on miss (book page 117)

Critical word first: send the word needed before fetching the whole block

Early restart: fetch the block and once the word that is needed is found forward it to the CPU

Merging write buffer: The write buffer holds the write instructions to the main memory (slow). Instead of writing individual instructions it can merge them and do them together.

Terminology

- Way predicting: In set associative cache predict the position of the multiplexer reading one of the positions in the set.
- Pipelined caches: Allow for execution of cache access instructions with one cycle difference 
- Non blocking caches: Allow hits before previous misses complete
- Hardware prefetching: On miss fetch 2 instead of 1 block. (the correct block and the one next to it)

Snooping Examples

time	Thread 1	Thread 2
t_1	read A	
t_2	write A	
t_3		read B
t_4		write B

time	Bus request	state A in P_1 (cache of P_1: processor 1)	State B in P_2 (cache of P_2: processor 2)
t_0		I	I
t_1	read miss A	S	I
t_2	invalidate	M	I
t_3	read miss B	M	S
t_4	invalidate	M	M

Example 1

time	Thread 1	Thread 2	Thread 3
t_1	read A		
t_2	write A		
t_3		read A	
t_4			read A

time	State A in P_1	State A in P_2	State A in P_3
t_0	I	I	I
t_1	S	I	I
t_2	M	I	I
t_3	S	S	I
t_4	S	S	S

Example 2

time	t_0	t_1	t_2	t_3	t_4
Bus Request		read miss	invalidate	read miss	read miss

Ιούνιος 2025, Ερώτηση 3

Instruction	Issue		Exe Start		Exe End		Write-Back	
LD F2, 0(R1)	1	1	2	2	2	2	3	3
LD F4, 0(R2)	2	2	4	4	4	4	5	5
MUL.D F6, F2, F4	3	3	6	6	9	9	10	10
LD F8, 0(R3)	4	4	6	6	6	6	7	7
ADD.D F10, F6, F8	5	5	11	11	12	12	13	13
SUB.D F12, F10, F4	6	6	14	14	15	15	16	16

Λειτουργικές μονάδες:	Πλήθος:	Πλήθος:
▷ Load (1 κύκλος)	1	1
▷ Mult (4 κύκλοι)	1	2
▷ Add/Sub (2 κύκλοι)	1	2

Reservation stations:	Πλήθος:
▷ Load	5
▷ Mult	2
▷ Add/Sub	3

Λόγω εξάρτησης και έλλειψης παραλληλοποιήσιμων εντολών οι δύο εκδοχές έχουν ίδιο Tomasulo.

Ιούνιος 2025, Ερώτημα 1

120s μονόπυρος επεξεργαστής

{ 80% παραλληλοποιήσιμο
{ 20% σειριακό

Speedup για χρήση αριθμού πυρήνων:

Χρόνος εκτέλεσης:

② : $sp = \frac{1}{0,2 + \frac{0,8}{2}} = \frac{1}{0,6} = 1,\bar{6}$

$1 \rightarrow 120s$, $0,6 \rightarrow x$ $\Big\}$ $x = 120 \cdot 0,6 = 72s$

④ : $sp = \frac{1}{0,2 + \frac{0,8}{4}} = \frac{1}{0,4} = 2,5$

$1 \rightarrow 120s$, $0,4 \rightarrow x$ $\Big\}$ $x = 120 \cdot 0,4 = 48s$

⑧ : $sp = \frac{1}{0,2 + \frac{0,8}{8}} = \frac{1}{0,3} = 3,\bar{3}$

$1 \rightarrow 120s$, $0,3 \rightarrow x$ $\Big\}$ $x = 120 \cdot 0,3 = 36s$

Όταν ο αριθμός των πυρήνων τείνει στο άπειρο έχουμε:

(∞): $sp = \frac{1}{0,2 + \frac{0,8}{\infty}} = \frac{1}{0,2+0} = 5$

$\left.\begin{matrix} 1 \rightarrow 120 \\ 0,2 \rightarrow x \end{matrix}\right\} x = 120 \cdot 0,2 = 24s$

Έξω στους 4 πυρήνες έχουμε 5% επιβράδυνση λόγω επικοινωνίας και συγχρονισμού:

Ο χρόνος που παίρνει μόνο ο παραλληλοποιημένος κώδικας 4 core είναι:

$48 - 24 = 24s$

χρόνος εκτέλεσης στις 4 πυρήνες

χρόνος εκτέλεσης που χρειάζεται το σειριακό κομμάτι κώδικα (το βρήκαμε βάζοντας ∞ αριθμό πυρήνων)

Άρα $24 \cdot 1,05 = 25,2s$ ο παραλληλοποιημένος 4πύρηνος κώδικας

$25,2 + 24 = 49,2s$

$\left.\begin{matrix} 1 \rightarrow 120 \\ x \rightarrow 49,2 \end{matrix}\right\} x = 0,41$ Δηλαδή: $sp = \frac{1}{0,41} = 2,4390$

Ιούνιος 2025, Ερώτημα 2

cache	size	hit time	miss rate
L_1 cache	32KB	1 cycle	5%
L_2 cache	256KB	10 cycles	20%
L_2 cache new	512KB	12 cycles	10%

Main memory hit time = 100 cycles

$AMAT_{L_1} = \text{hit time}_{L_1} + \text{miss rate}_{L_1} \cdot \text{miss penalty}_{L_1}$

$\text{miss penalty}_{L_1} = AMAT_{L_2} = \text{hit time}_{L_2} + \text{miss rate}_{L_2} \cdot \text{miss penalty}_{L_2}$

$= 10 + (0,2 \cdot 100)$

$= 30$

$AMAT_{L_1} = 1 + 0,05 \cdot 30 = 2,5$ cycles

$AMAT_{L_1} = 10 + 0,05 \cdot (12 + 0,1 \cdot 100) = 2,1$

Και με αμφίβολη η απάντηση διότι πηγαίνω πιο σπάνια στην κύρια μνήμη της οποίας το hit time είναι πολύ μεγάλο

Έστω $2 \cdot 10^9$ εντολές με ιδανικό CPI = 1. Υπολογίστε την αλλαγή στο συνολικό χρόνο εκτέλεσης.

1η έκδοση: AMAT = 2,5 ⇒ Total cycles = $2,5 \cdot 2 \cdot 10^9 = 5 \cdot 10^9$

2η έκδοση: AMAT = 2,1 ⇒ Total cycles = $2,1 \cdot 2 \cdot 10^9 = 4,2 \cdot 10^9$

} $5 \cdot 10^9 - 4,2 \cdot 10^9 = 0,8 \cdot 10^9$

Εξοικονομούν $0,8 \cdot 10^9$ κύκλους